

SQL

Foundations

How to run it in docker container

1.Run docker container

```
docker run --name postgres-db -e POSTGRES_PASSWORD=pass -p 5432:5432 postgres
```

2. Go inside container

```
docker exec -it postgres-db bash
```

3. Enter PostgreSQL CLI

```
psql -U postgres
```

4.Start writing sql queries

Creating Database

```
CREATE DATABASE devhub;
```

Create a new db

1 Tables

A **table** is where data is stored in a relational database.

Think of it like a **spreadsheet**.

Example **users table**

id	name	email
1	Prajwal	p@gmail.com
2	Rahul	r@gmail.com

Important ideas

Row → one record

Column → property of that record

Example meaning:

Row = one user

Column = attribute of user

Create Your First Table

Now we create **users table**.

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

What this means:

Part	Meaning
SERIAL	auto increment id
PRIMARY KEY	unique identifier
TEXT	string
UNIQUE	no duplicate emails
NOT NULL	must provide value

Primary Key

A **Primary Key** uniquely identifies every row in a table.

Rules:

- Cannot repeat
- Cannot be NULL
- Every table should have one

Backend frameworks **depend heavily on primary keys**.

4. Foreign Key

A **Foreign Key** connects two tables.

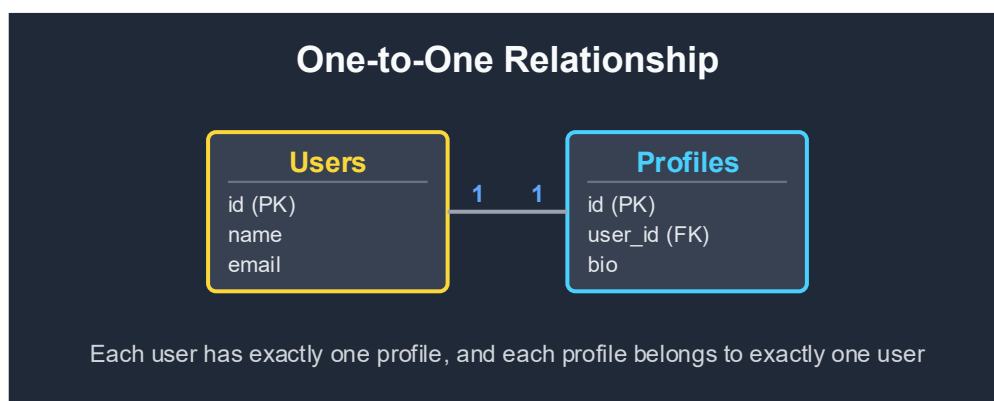
```
user_id INTEGER UNIQUE REFERENCES users(id), // FOREIGN KEY POINTS TO  
TABLE USERS → ID COLUMN
```

RELATIONSHIPS

1 One-to-One Relationship

One record in table A corresponds to exactly one record in table B.

```
CREATE TABLE user_profiles (  
id SERIAL PRIMARY KEY,  
user_id INTEGER UNIQUE REFERENCES users(id),  
bio TEXT,  
avatar TEXT  
);
```



2 One-to-Many (Most Important)

Meaning:

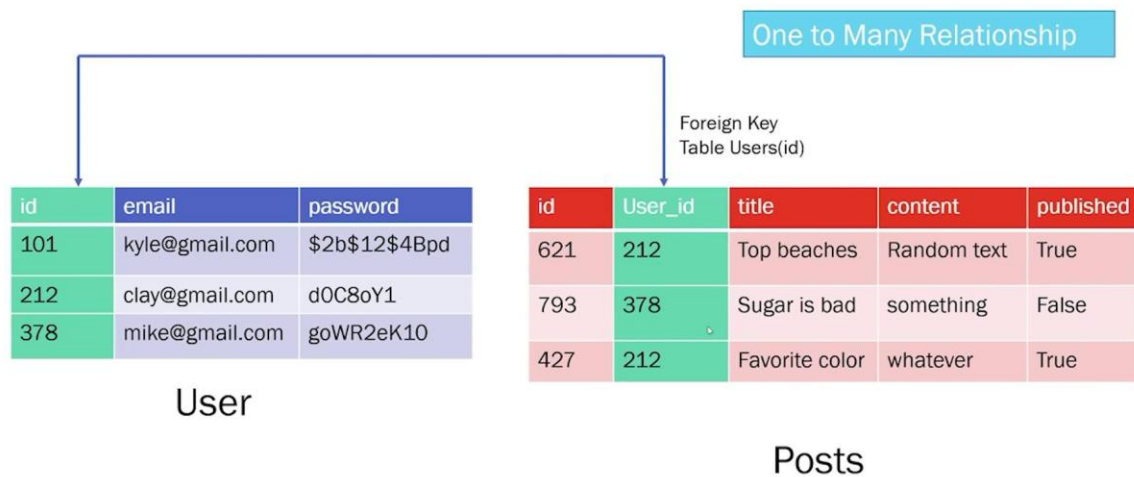
One user → many posts

One post → many comments

One user → many orders

```
CREATE TABLE posts (  
  id SERIAL PRIMARY KEY,  
  title TEXT,  
  user_id INTEGER REFERENCES users(id)  
);
```

Relationships



⏪ ⏴ ⏵ ⏩ 🔍 🔄 🗑️

3 Many-to-Many

Meaning:

Many students → many courses

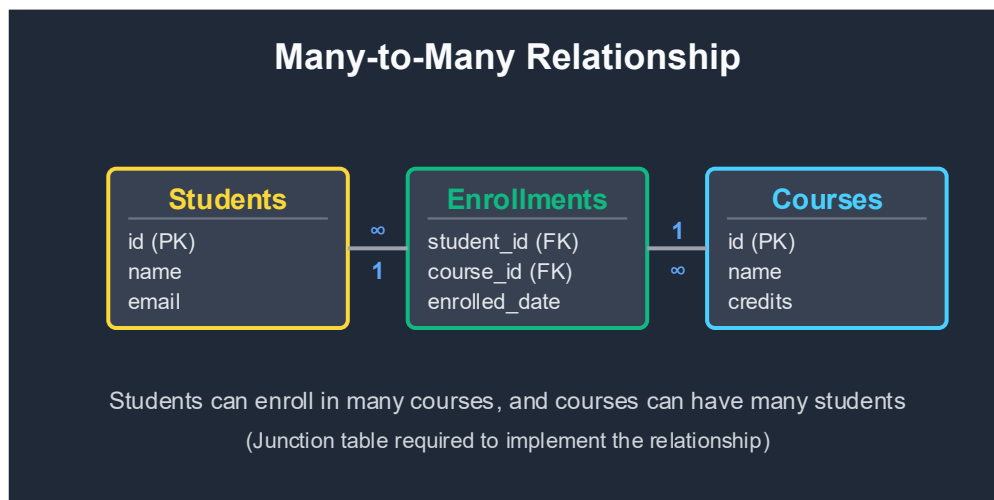
Many posts → many tags

Many users → many roles

You **cannot connect them directly**.

You must create a **junction table**.

```
CREATE TABLE student_courses (  
  student_id INTEGER REFERENCES students(id),  
  course_id INTEGER REFERENCES courses(id),  
  PRIMARY KEY (student_id, course_id)  
);
```



Real Backend Example (Very Typical)

Most social platforms look like this:

users
posts
comments
likes

Relationships:

users → posts (one-to-many)
posts → comments (one-to-many)
users → comments (one-to-many)
users ↔ posts (many-to-many via likes)

The Backend Developer Mental Model

When designing a database, you ask **three questions**:

- 1 Is it **one-to-one**?
- 2 Is it **one-to-many**?
- 3 Is it **many-to-many**?

That's basically **90% of schema design**.

Use **One-to-One** when:

- data is optional
- data is large
- data belongs to same entity but different concern
- you want cleaner tables

So the decision becomes:

👉 **Keep it in same table OR split into another table?**

If **one entity naturally creates many others**, use **One-to-Many**.

If both sides can have many

👉 **Create an intermediate table**

Core SQL operation

1.INSERT

1 Insert One User

```
INSERT INTO users (name, email)
VALUES ('Prajwal Jadhav', 'prajwal@devhub.com');
```

What happens:

- id → automatically generated
 - created_at → automatically set to current time
-

2 Insert Multiple Users (More Realistic)

Backend systems usually insert many users.

```
INSERT INTO users (name, email)
VALUES
('Alex Morgan', 'alex@devhub.com'),
('Sarah Lee', 'sarah@devhub.com'),
('David Kim', 'david@devhub.com'),
('Ananya Sharma', 'ananya@devhub.com');
```

2.SELECT Get Data

The simplest form just pulls data from a table.

```
SELECT * FROM users;
```

Meaning:

SELECT → what data you want

* → all columns

FROM → which table

Example output:

id	name	email
1	Prajwal	prajwal@email.com
2	Alex	alex@email.com

2 Selecting Specific Columns

You usually **don't fetch everything** in real apps.

```
SELECT name, email FROM users;
```

Output:

name	email
Prajwal	prajwal@email.com
Alex	alex@email.com

Why?

- faster queries
- less data transfer
- cleaner API responses

3 WHERE — Filter Data

This is how you **search or filter** records.

```
SELECT * FROM users  
WHERE id = 1;
```

Result:

id	name	email
1	Prajwal	prajwal@email.com

Another example:

```
SELECT * FROM users  
WHERE name = 'Prajwal';
```


4 Multiple Conditions

```
SELECT * FROM users  
WHERE id > 1;
```

Operators you'll use a lot:

- = equal
- != not equal
- > greater
- < less
- >= greater equal
- <= less equal

Example:

```
SELECT * FROM users  
WHERE id >= 2;
```

5 AND / OR

Filter with multiple rules.

```
SELECT * FROM users  
WHERE name = 'Prajwal'  
AND id = 1; // both should true
```

OR example:

```
SELECT * FROM users  
WHERE name = 'Prajwal'  
OR name = 'Alex'; // only one should true
```

6 ORDER BY — Sorting

```
SELECT * FROM users  
ORDER BY name;
```

Ascending (default).

Example result:

Alex
Prajwal
Sam

Descending:

```
SELECT * FROM users  
ORDER BY name DESC;
```

Example result:

Sam
Prajwal
Alex

7 LIMIT — Restrict Results

Very important for **pagination**.

```
SELECT * FROM users  
LIMIT 2;
```

Output:

id	name
1	Prajwal
2	Alex

8 OFFSET — Pagination

```
SELECT * FROM users  
LIMIT 2 OFFSET 2;
```

Meaning:

skip first 2 rows
then return next 2

Used in APIs:

GET /users?page=2

9 LIKE — Search

```
SELECT * FROM users  
WHERE name LIKE 'P%';
```

Meaning:

names starting with P

Examples:

Prajwal

Pratik

Priya

10 COUNT — Aggregate

```
SELECT COUNT(*) FROM users;
```

Result:

3

Very common in dashboards and analytics.

UPDATE

Basic UPDATE Syntax

```
UPDATE table_name  
SET column = value  
WHERE condition;
```

Meaning:

- UPDATE → which table to change
- SET → what column(s) to change
- WHERE → which rows should be updated

Example:

```
UPDATE users  
SET name = 'Prajwal J'  
WHERE id = 1;
```

This updates the name of the user whose id = 1.

2 Updating Multiple Columns

You can update more than one field at once.

```
UPDATE users  
SET name = 'Prajwal J',  
    email = 'prajwalj@email.com'  
WHERE id = 1;
```

Now both name and email change together.

3 Updating Using Another Condition

Example: update based on name.

```
UPDATE users  
SET email = 'new@email.com'  
WHERE name = 'Prajwal';
```

If there are multiple rows named Prajwal, all of them will be updated.

VERY IMPORTANT: Without WHERE

If you run this:

```
UPDATE users  
SET name = 'Test';
```

 Every row in the table will change.

Updating Using Math / Expressions

Example: increase a value.

Imagine a table:

```
products  
id | name | price
```

Increase price:

```
UPDATE products  
SET price = price + 100  
WHERE id = 1;
```

This reads the existing value and modifies it.

Returning Updated Rows (PostgreSQL feature)

Postgres has a very nice feature:

```
UPDATE users  
SET name = 'Prajwal'  
WHERE id = 1  
RETURNING *;
```

It immediately returns the updated row.

Backend frameworks love this because it saves another query.

DELETE

1 Basic DELETE Syntax

DELETE FROM table_name
WHERE condition;

Meaning:

- DELETE FROM → which table
- WHERE → which rows should be removed

Example:

DELETE FROM users
WHERE id = 3;

This removes the user whose id = 3.

2 Delete Using Another condition

Example: remove all users named Sam.

DELETE FROM users
WHERE name = 'Sam';

If multiple rows match → all of them will be deleted.

3 VERY IMPORTANT: DELETE Without WHERE ⚠

DELETE FROM users;

This deletes every row in the table.

Backend dev rule:

Never run DELETE without WHERE unless you really intend to clear everything.

4 Delete and Return Rows (PostgreSQL feature)

Postgres lets you see what was deleted:

```
DELETE FROM users  
WHERE id = 2  
RETURNING *;
```

Output:

id name email

2 Alex a@email.com

This is very useful for logging or debugging.

5 Delete Multiple Rows

Example:

```
DELETE FROM users  
WHERE id > 5;
```

This removes all users whose id is greater than 5.

6 Safe Deletion Pattern (Real Developer Habit)

Good developers check before deleting.

Step 1 — preview rows:

```
SELECT * FROM users  
WHERE name = 'Sam';
```

Step 2 — delete:

```
DELETE FROM users  
WHERE name = 'Sam';
```

This prevents deleting the wrong data.

7 Delete With Relationships (Foreign Keys)

Suppose you have:

users

posts

posts table:

id

title

user_id

If you try:

DELETE FROM users

WHERE id = 1;

Postgres may stop you if posts exist:

ERROR: update or delete violates foreign key constraint

Because posts depend on that user.

Solutions developers use:

Option 1 — Delete dependent rows first

DELETE FROM posts WHERE user_id = 1;

DELETE FROM users WHERE id = 1;

Option 2 — Use cascade delete

When designing the table:

user_id INTEGER REFERENCES users(id) ON DELETE CASCADE

Then deleting a user automatically deletes their posts.

8 Difference: DELETE vs TRUNCATE

Both remove data, but they behave differently.

DELETE

DELETE FROM users;

- removes rows one by one
 - can use WHERE
 - slower
 - safer
-

TRUNCATE

TRUNCATE TABLE users;

- removes all rows instantly
- very fast
- cannot filter
- resets auto-increment ids

Aggregations

1 COUNT — Count Rows

Counts how many rows exist.

Query

```
SELECT COUNT(*)  
FROM users;
```

Result:

count

3

Meaning:

Total rows in users table

Count with condition

```
SELECT COUNT(*)  
FROM users  
WHERE name = 'Prajwal';
```

Counts only rows matching the condition.

Count specific column

```
SELECT COUNT(email)  
FROM users;
```

2 SUM — Add Numbers

```
SELECT SUM(amount)  
FROM orders;
```

Gives sum of all orders from amount column.

```
SELECT SUM(amount)
FROM orders
WHERE user_id = 1;
```

Give sum of orders of user id 1 from amount column.

3 AVG — Average

Calculates the mean value.

Example:

```
SELECT AVG(amount)
FROM orders;
```

Calculation:

$(100 + 200 + 150) / 3$

Result:

150

4 GROUP BY — Group Data

This is where aggregation becomes powerful.

GROUP BY means:

Group rows that have the same value
then apply aggregation

Query

```
SELECT user_id, SUM(amount)
FROM orders
GROUP BY user_id;
```

Example: Orders table

id	user_id	amount
1	1	100
2	1	200
3	2	150

Group orders by user

Then calculate total spending per user

Result:

user_id	sum
1	300
2	150

5 GROUP BY with COUNT

Example:

```
SELECT name, COUNT(*)  
FROM users  
GROUP BY name;
```

Result:

name	count
Prajwal	2
Alex	1

Meaning:

How many users share the same name

6 GROUP BY + ORDER BY

Example: top spenders.

```
SELECT user_id, SUM(amount) AS total_spent  
FROM orders  
GROUP BY user_id  
ORDER BY total_spent DESC;
```

Result:

user_id	total_spent
1	300
2	150

JOINS

A JOIN combines rows from two or more tables based on a related column between them, usually a primary key and a foreign key.

SQL JOIN — General Syntax

```
SELECT table1.column1, table2.column2
FROM table1
JOIN table2
ON table1.foreign_key = table2.primary_key;
```

Explanation

1 SELECT

```
SELECT table1.column1, table2.column2
```

Tells the database what data you want to display.

Example:

```
SELECT posts.title, users.name
```

Meaning:

Show the post title

Show the user's name

2 FROM

```
FROM table1
```

Tells the database which table to start with.

Example:

```
FROM posts
```

Meaning:

Start reading rows from the posts table

3 JOIN

```
JOIN table2
```

Tells SQL to bring another table into the query.

Example:

JOIN users

Meaning:

Also use data from the users table

 **ON (The Matching Rule)**

ON table1.foreign_key = table2.primary_key

This tells SQL how rows from the two tables connect.

Example:

ON posts.user_id = users.id

Meaning:

Match each post with the user whose id equals user_id

Simple Mental Model

JOIN means:

Connect rows from two tables using matching IDs

Most joins follow this pattern:

table1.foreign_key = table2.primary_key

Example:

posts.user_id → users.id

Example Query

SELECT posts.title, users.name

FROM posts

JOIN users

ON posts.user_id = users.id;

Result:

title

name

Docker Guide

Prajwal

title

name

SQL Basics

Prajwal

Hello World

Alex

Short Version for Quick Revision

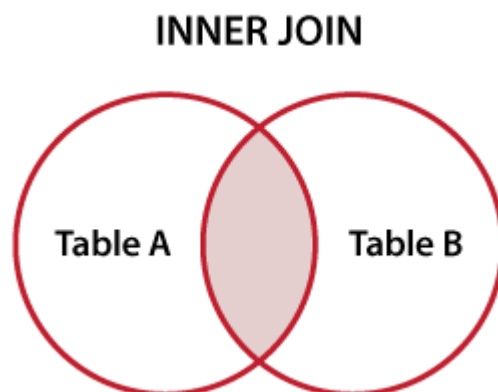
SELECT → what columns to show

FROM → first table

JOIN → second table

ON → condition that connects the tables

1. INNER JOIN



Returns **only rows that have matching values in both tables.**

```
SELECT users.name, posts.title
```

```
FROM users
```

```
INNER JOIN posts
```

```
ON users.id = posts.user_id;
```


Result

name	title
Prajwal	Docker Guide
Prajwal	SQL Basics
Alex	Hello World

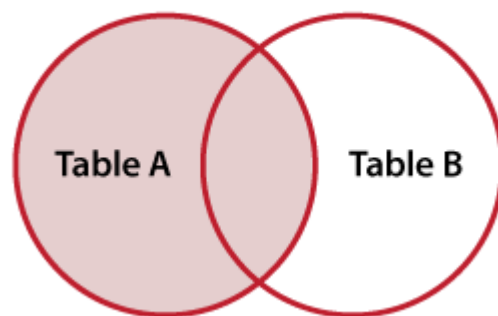
Sam is excluded because **he has no posts**.

2. LEFT JOIN

Returns **all rows from the left table and matching rows from the right table**.

If no match exists, the result contains **NULL values**.

LEFT OUTER JOIN



```
SELECT users.name, posts.title  
FROM users  
LEFT JOIN posts  
ON users.id = posts.user_id;
```

Result

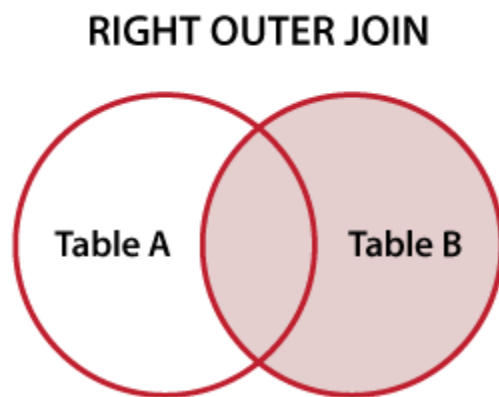
name	title
Prajwal	Docker Guide
Prajwal	SQL Basics

name	title
Alex	Hello World
Sam	NULL

Sam appears because **LEFT JOIN keeps every user**.

Mental model: Everything from LEFT table + matching rows

3. RIGHT JOIN



Returns **all rows from the right table and matching rows from the left table**.

If no match exists, the left side becomes **NULL**.

Query

```
SELECT users.name, posts.title
FROM users
RIGHT JOIN posts
ON users.id = posts.user_id;
```

Mental model:

Everything from RIGHT table + matching rows

4. FULL JOIN (FULL OUTER JOIN)

Interview Definition

A FULL JOIN returns all rows from both tables.

If a match exists, the rows are combined.

If no match exists, NULL values are filled for the missing side.

Syntax

```
SELECT users.name, posts.title  
FROM users  
FULL JOIN posts  
ON users.id = posts.user_id;
```

Result

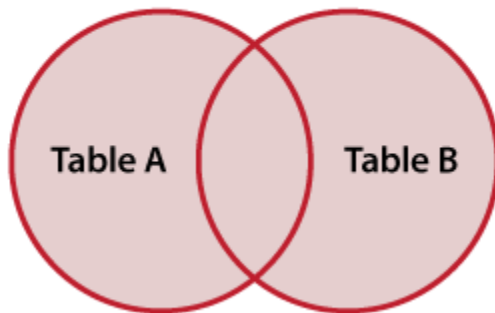
name	title
Prajwal	Docker Guide
Prajwal	SQL Basics
Alex	Hello World
Sam	NULL
NULL	Unknown Post

Explanation:

- Sam → user without post
- Unknown Post → post without user

FULL JOIN keeps both sides.

FULL OUTER JOIN



SQL JOIN — Mental Model Cheat Sheet

JOIN Type	Mental Model	Returns
INNER JOIN	Only the overlap	Only rows that match in both tables
LEFT JOIN	Left table is the host	All rows from left + matching rows from right
RIGHT JOIN	Right table is the host	All rows from right + matching rows from left
FULL JOIN	Everything from both sides	All rows from both tables, matched where possible

INDEXES

An index is a database structure that improves the speed of data retrieval operations on a table by allowing the database to locate rows quickly without scanning the entire table.

Think of a book index.

Without index:

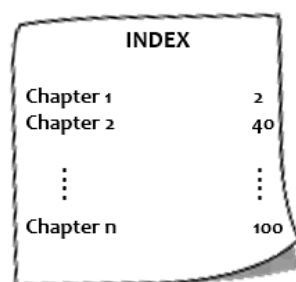
Search page by page

$O(n)$

With index:

Jump directly to location

$O(\log n)$

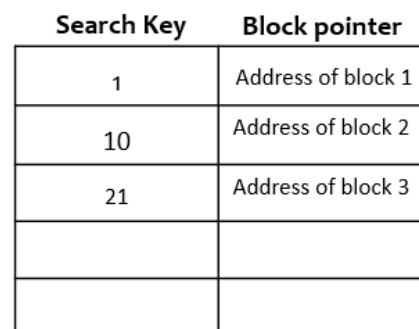


A hand-drawn sketch of a book index page. It is titled 'INDEX' at the top. Below the title, it lists 'Chapter 1' with the page number '2', 'Chapter 2' with '40', three vertical dots, and 'Chapter n' with '100'.

INDEX	
Chapter 1	2
Chapter 2	40
⋮	⋮
Chapter n	100

Index page of a book

Analogy →



A table representing a database index file. It has two columns: 'Search Key' and 'Block pointer'. The first three rows contain the values 1, 10, and 21 in the 'Search Key' column, and 'Address of block 1', 'Address of block 2', and 'Address of block 3' in the 'Block pointer' column. The last two rows are empty.

Search Key	Block pointer
1	Address of block 1
10	Address of block 2
21	Address of block 3

Index file of a database file

Indexes improve **read speed**, but slow **writes** (INSERT/UPDATE).

So use them **carefully**.

Basic Syntax

```
CREATE INDEX index_name  
ON table_name (column_name);
```

When to Use Index

Create indexes on columns used in:

WHERE

JOIN

ORDER BY

Example query:

```
SELECT * FROM users
```

```
WHERE email = 'test@email.com';
```

Index helps this query run **much faster**.

Primary keys & unique attribute automatically create an index.

Transactions

What is a Transaction?

A transaction is a group of database operations executed as a single unit.

Meaning:

Either all operations succeed

OR none of them happen

This keeps the database consistent.

Real Example (Bank Transfer)

Suppose Prajwal sends ₹500 to Alex.

Two operations happen:

1 Deduct ₹500 from Prajwal

2 Add ₹500 to Alex

If step 1 happens but step 2 fails → money disappears ❌

Transactions prevent this.

Basic Syntax

Start transaction

`BEGIN;`

Run queries

`UPDATE accounts SET balance = balance - 500 WHERE name = 'Prajwal';`

`UPDATE accounts SET balance = balance + 500 WHERE name = 'Alex';`

Finish transaction

`COMMIT;`

If Something Fails

Cancel everything:

`ROLLBACK;`

Transactions follow **ACID properties**:

Property	Meaning
Atomicity	all or nothing
Consistency	data stays valid
Isolation	transactions don't interfere
Durability	committed data is permanent

What Are Constraints?

Constraints = rules applied to table columns to ensure data integrity.

Example rules:

- Email must be unique
- Name cannot be empty
- ID must be unique
- Post must belong to a valid user

Most Important SQL Constraints (Backend Developers Use These Daily)

1 NOT NULL

Means a column **must contain a value**.

Means a column **must contain a value**.

Example:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT NOT NULL  
);
```

Now this will **FAIL**:

```
INSERT INTO users (email)  
VALUES ('test@email.com');
```

Error because name is required.

Backend benefit:

- Prevents incomplete records.

2 UNIQUE

Ensures **no duplicate values**.

Perfect for:

- emails
- usernames
- phone numbers

Example:

email TEXT UNIQUE

Now this works:

```
INSERT INTO users (name,email)
VALUES ('Alex','alex@email.com');
```

But this fails:

```
INSERT INTO users (name,email)
VALUES ('John','alex@email.com');
```

Database says:

✗ duplicate key value violates unique constraint

4 DEFAULT

Automatically adds a value.

Example:

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Now when inserting:

```
INSERT INTO users (name,email)
VALUES ('Sam','sam@email.com');
```

Database automatically sets:

created_at = now()

Very useful in production.

5 CHECK Constraint

Adds **custom rule validation**.

Example:

```
age INT CHECK (age >= 18)
```

Now this fails:

```
INSERT INTO users (age)  
VALUES (15);
```

Database rejects it.

Query Optimization

making SQL queries run faster and use fewer resources.

When a query runs, the database must:

1. Find the data
2. Filter it
3. Join tables
4. Return results

If the database has to **scan the entire table**, it becomes slow.

Goal: **help the database find data quickly.**

1 Indexes (The #1 Optimization Tool)

2 Avoid SELECT *

3 Use WHERE Early

Always filter as early as possible.

4 Use LIMIT (Pagination)

Returning huge datasets is slow.

5 Avoid N+1 Query Problem

This is a **very common backend mistake.**

Bad backend logic:

1 query → get users

100 queries → get posts for each user

Total:

101 queries

Better solution → **JOIN**

```
SELECT users.name, posts.title  
FROM users
```

JOIN posts

ON users.id = posts.user_id;

Now only **1 query**.

Huge performance gain.